

Настройка уведомлений «Символики»

Краткая карта всех токенов, правила хранения и обязательная ротация перед релизом: [Токены и секреты](#).

Инструкция относится к текущей реализации системы и описывает два независимых контура:

1. уведомления сотрудникам о заказах, позициях, задачах, производстве, закупках, письмах и финансах;
2. автоматическое уведомление клиента «Заказ готов и находится в офисе».

SMS в текущей версии не реализованы. Центр уведомлений внутри системы работает без внешних провайдеров. Push, Email, ВКонтакте и Telegram требуют отдельной настройки.

1. Что подготовить

Понадобятся:

- публичный HTTPS-адрес системы, например `https://account.example.ru`;
- пара VAPID-ключей для браузерных Push;
- ключ доступа сообщества ВКонтакте;
- Telegram-бот и его токен;
- параметры SMTP почты `start@symb62.ru` на REG.RU;
- доступ к файлу `.env` рядом с `docker-compose.yml` на сервере.

Не пересылайте токены и пароли в чатах, не добавляйте их в Git и не вставляйте в скриншоты.

2. Обязательная подготовка `docker-compose.yml`

В текущем файле исторически присутствуют встроенные значения Push и VK. Перед рабочим запуском:

1. перевыпустите ключ сообщества ВКонтакте;
2. выпустите новую пару VAPID-ключей;
3. замените значения соответствующих строк в `environment` сервиса Directus на ссылки на `.env`:

```
SYMBOLIKA_PUBLIC_URL: "${SYMBOLIKA_PUBLIC_URL:-http://localhost:8057}"
SYMBOLIKA_PUSH_PUBLIC_KEY: "${SYMBOLIKA_PUSH_PUBLIC_KEY:-}"
SYMBOLIKA_PUSH_PRIVATE_KEY: "${SYMBOLIKA_PUSH_PRIVATE_KEY:-}"
SYMBOLIKA_PUSH_SUBJECT: "${SYMBOLIKA_PUSH_SUBJECT:-mailto:start@symb62.ru}"
SYMBOLIKA_VK_TOKEN: "${SYMBOLIKA_VK_TOKEN:-}"
SYMBOLIKA_VK_API_VERSION: "${SYMBOLIKA_VK_API_VERSION:-5.199}"
SYMBOLIKA_VK_PRODUCTION_PEER_ID: "${SYMBOLIKA_VK_PRODUCTION_PEER_ID:-}"
SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID: "${SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID:-}"
```

Telegram и SMTP уже должны использовать аналогичные ссылки на `.env`. Не публикуйте результат команды `docker compose config`: он может содержать подставленные секреты.

3. Заполнить `.env` на сервере

Откройте каталог установки и создайте `.env`, если его ещё нет:

```
cd /path/to/symbolika_directus_clean_install
cp -n .env.example .env
nano .env
```

Заполните параметры:

```
# Публичный адрес системы. На сервере обязательно HTTPS.
SYMBOLIKA_PUBLIC_URL=https://YOUR_DOMAIN

# Push
SYMBOLIKA_PUSH_PUBLIC_KEY=
SYMBOLIKA_PUSH_PRIVATE_KEY=
SYMBOLIKA_PUSH_SUBJECT=mailto:start@symb62.ru

# ВКонтакте
SYMBOLIKA_VK_TOKEN=
SYMBOLIKA_VK_API_VERSION=5.199
SYMBOLIKA_VK_PRODUCTION_PEER_ID=
SYMBOLIKA_VK_SCREEN_PRINTING_PEER_ID=

# Telegram
SYMBOLIKA_TELEGRAM_BOT_TOKEN=

# Отправка Email через REG.RU
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=start@symb62.ru
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <start@symb62.ru>"
```

Параметры IMAP встроенного почтового клиента настраиваются отдельно по инструкции [Настройка встроенной почты](#). Для уведомлений достаточно SMTP-параметров выше.

4. Настроить браузерные Push

4.1. Сгенерировать VAPID-ключи

Ключи создаются один раз и затем постоянно используются сервером. Например:

```
npx web-push generate-vapid-keys --json
```

Скопируйте `publicKey` в `SYMBOLIKA_PUSH_PUBLIC_KEY`, а `privateKey` — в `SYMBOLIKA_PUSH_PRIVATE_KEY`. Закройте терминал после копирования и не сохраняйте его вывод в общий журнал.

`SYMBOLIKA_PUSH_SUBJECT` должен быть адресом вида `mailto:start@symb62.ru` либо публичным HTTPS URL.

4.2. Подключить устройство сотрудника

После запуска системы каждый сотрудник выполняет это на каждом нужном компьютере или телефоне:

1. войти в систему;
2. открыть иконку колокольчика `Центр уведомлений`;
3. открыть вкладку `Каналы доставки`;
4. включить `Push` в браузере;
5. нажать `Подключить это устройство`;
6. разрешить уведомления в диалоге браузера;
7. нажать `Сохранить настройки`.

На сервере Push требуют HTTPS. `localhost` является допустимым исключением только для локальной проверки. Если пользователь запретил уведомления, разрешение нужно вернуть в настройках сайта самого браузера.

5. Настроить Telegram

5.1. Создать бота

1. Откройте официального бота `@BotFather`.
2. Выполните `/newbot`.
3. Укажите название и username, заканчивающийся на `bot`.
4. Сохраните полученный токен в `SYMBOLIKA_TELEGRAM_BOT_TOKEN`.

Официальная документация: <https://core.telegram.org/bots/tutorial>.

5.2. Получить `chat_id`

Бот не может первым начать личный диалог. Сотрудник или клиент должен открыть бота и нажать `Запустить` либо отправить `/start`.

Для разовой настройки получите `chat.id`:

```

$token = Read-Host "Telegram bot token"
$updates = Invoke-RestMethod "https://api.telegram.org/bot$token/getUpdates"
$updates.result | ForEach-Object {
    [PSCustomObject]@{
        chat_id = $_.message.chat.id
        username = $_.message.chat.username
        name = ($_.message.chat.first_name, $_.message.chat.last_name -join " ").Trim()
        text = $_.message.text
    }
}
$token = $null

```

Если у бота установлен webhook, `getUpdates` одновременно не работает. Telegram поддерживает либо webhook, либо long polling: <<https://core.telegram.org/bots/faq#getting-updates>>.

Для сотрудника `chat_id` вводится в Центр уведомлений → Каналы доставки → Telegram. Для клиента — в карточке клиента или компании.

6. Настроить ВКонтакте

6.1. Подготовить сообщество

1. В управлении сообществом включите сообщения.
2. В разделе API создайте ключ доступа сообщества с правом отправки сообщений.
3. Запишите новый ключ в `SYMBOLIKA_VK_TOKEN`.
4. Не используйте токен личной страницы.

Система отправляет сообщения методом `messages.send`, используя `peer_id` получателя.

6.2. Получить `peer_id`

Сотрудник или клиент сначала должен написать сообществу любое сообщение. Затем `peer_id` можно получить из диалогов сообщества:

```

$token = Read-Host "VK community token"
$result = Invoke-RestMethod -Method Post `
    -Uri "https://api.vk.com/method/messages.getConversations" `
    -Body @{ access_token = $token; v = "5.199"; count = 50 }
$result.response.items | ForEach-Object {
    [PSCustomObject]@{
        peer_id = $_.conversation.peer.id
        text = $_.last_message.text
    }
}
$token = $null

```

Для сотрудника `peer_id` вводится в Центр уведомлений → Каналы доставки → ВКонтакте . Для клиента — в карточке клиента или компании. Не вводите короткое имя страницы вида `vk.com/username` .

7. Настроить отправку Email через REG.RU

Для почтового хостинга REG.RU текущая конфигурация использует:

```
SYMBOLIKA_SMTP_HOST=mail.hosting.reg.ru
SYMBOLIKA_SMTP_PORT=465
SYMBOLIKA_SMTP_SECURE=true
SYMBOLIKA_SMTP_USER=start@symb62.ru
SYMBOLIKA_SMTP_PASSWORD=
SYMBOLIKA_EMAIL_FROM="Символика <start@symb62.ru>"
```

REG.RU указывает `mail.hosting.reg.ru` как сервер исходящей почты для защищённого соединения: <https://help.reg.ru/support/hosting/nastroyka-pochty-regru/nastroyka-pochty-i-pochtovykh-kliientovv/nastroyka-pochtovykh-kliientovv>.

Для нормальной доставки проверьте DNS домена:

- SPF разрешает серверу REG.RU отправлять письма от имени домена;
- DKIM включён в панели почтового хостинга;
- DMARC добавлен хотя бы в режиме мониторинга.

Сотрудник включает Email в своём центре уведомлений и указывает адрес. Если адрес не указан, сервер может использовать Email учётной записи Directus.

8. Применить настройки на сервере

Обычный `docker restart` не перечитывает `.env` . Контейнер Directus нужно пересоздать:

```
cd /path/to/symbolika_directus_clean_install
docker compose up -d --force-recreate directus
docker logs --tail 120 symbolika-directus
```

Проверьте наличие настроек, не выводя их значения:

```
docker exec symbolika-directus node -e '
const names = [
  "SYMBOLIKA_PUBLIC_URL",
  "SYMBOLIKA_PUSH_PUBLIC_KEY",
  "SYMBOLIKA_PUSH_PRIVATE_KEY",
  "SYMBOLIKA_VK_TOKEN",
  "SYMBOLIKA_TELEGRAM_BOT_TOKEN",
  "SYMBOLIKA_SMTP_HOST",
  "SYMBOLIKA_SMTP_USER",
  "SYMBOLIKA_SMTP_PASSWORD",
  "SYMBOLIKA_EMAIL_FROM"
];
for (const name of names) console.log(name + ": " + (process.env[name] ? "OK" : "НЕ
ЗАПОЛНЕНО"));
```

9. Персональная настройка сотрудника

Каждый сотрудник открывает **Центр уведомлений** → **Каналы доставки** и выбирает события:

- статусы заказов;
- позиции и макеты;
- новые задачи;
- изменения задач;
- производство;
- закупки;
- новые письма;
- финансы.

Критические системные события не отключаются. Для каждого внешнего канала нужно включить переключатель и указать получателя. После изменений нажать **Сохранить настройки**.

Уведомление внутри центра системы является основным. Внешние каналы доставляют его копию и ссылку на связанный заказ, позицию или задачу.

10. Настройка уведомления клиенту

Сейчас клиенту автоматически отправляется только сообщение:

Заказ готов и находится в офисе.

Другие статусы клиенту не отправляются.

10.1. Заполнить общие данные

Администратор открывает:

Нужно заполнить:

- адрес офиса;
- режим работы;
- сайт;
- ссылку на группу ВКонтакте.

10.2. Выбрать канал клиента

1. Откройте **Заказы → Клиенты и расчёты → Клиенты** или **Компании**.
2. Откройте карточку.
3. В блоке уведомления о готовом заказе выберите **Email**, **Telegram**, **ВКонтакте** или **Не отправлять**.
4. Укажите **Email**, **chat_id** или **peer_id**.

Если в заказе выбраны клиент и компания, сначала используется канал клиента. Настройки компании применяются, если у клиента выбран вариант **Не отправлять**.

10.3. Условия автоматической отправки

Сообщение уйдёт только когда одновременно выполнено всё:

1. заказ имеет статус **Готов**;
2. способ получения — **выдача в офисе**;
3. офисный статус заказа — **В офисе**;
4. все позиции находятся в офисе;
5. у клиента или компании выбран канал;
6. заполнен получатель выбранного канала.

В сообщение включаются позиции, количество, цены, сумма, оплачено, остаток, адрес и режим работы офиса. Успешно отправленное сообщение повторно по тому же заказу не отправляется.

11. Пошаговая проверка

Проверка сотрудника

1. Создайте тестового сотрудника или используйте свой аккаунт.
2. Подключите один канал в центре уведомлений.
3. Включите тему **Новые задачи**.
4. Назначьте этому сотруднику новую задачу с другого аккаунта.
5. Проверьте запись в центре и внешнее сообщение.
6. В центре возле внешнего канала должен появиться статус **доставлено** или **ошибка**.

Проверка клиента

1. Создайте тестового клиента со своим **Email**, **chat_id** или **peer_id**.

2. Создайте заказ с выдачей в офисе.
3. Добавьте позицию и доведите её до готовности.
4. Установите офисный статус `В офисе`.
5. Проверьте сообщение и журнал клиентских уведомлений.

Проверка логов

```
docker logs --tail 200 symbolika-directus 2>&1 | grep -Ei
"notification|telegram|vk|smtp|push|send failed"
```

В логи не должны попадать сами токены и пароли.

12. Частые ошибки

Симптом	Что проверить
Push сообщает, что ключ не настроен	VAPID public/private присутствуют в окружении пересозданного контейнера
Браузер не спрашивает разрешение	HTTPS, разрешения сайта, поддержка Service Worker и Push API
Telegram <code>chat not found</code>	пользователь запустил бота; введён правильный <code>chat_id</code>
<code>getUpdates</code> не возвращает сообщения	у бота установлен webhook либо пользователь ещё не написал боту
VK не отправляет	сообщения сообщества включены; клиент начал диалог; токен сообщества и <code>peer_id</code> верны
SMTP authentication failed	логин, пароль ящика, порт 465 и защищённое соединение
В центре есть уведомление, внешнего сообщения нет	канал выключен, не указан получатель либо провайдер вернул ошибку
Клиентское уведомление не создано	не выполнено одно из условий готовности и нахождения в офисе
Уведомление имеет статус <code>sent</code> , но повторно не приходит	штатно сработала защита от дублей

13. Рекомендуемый порядок включения

1. Перевыпустить и вынести секреты из `docker-compose.yml` в `.env`.
2. Настроить `SYMBOLIKA_PUBLIC_URL` и HTTPS.
3. Подключить SMTP и проверить Email.
4. Настроить Telegram-бота и тестовый `chat_id`.
5. Настроить сообщество VK и тестовый `peer_id`.
6. Выпустить VAPID-ключи и подключить Push на одном устройстве.
7. Провести тест назначения задачи сотруднику.

8. Провести тест готового клиентского заказа в офисе.

9. Подключать сотрудников и клиентов постепенно, проверяя журнал доставки.

VAPID-ключи следует создать один раз и хранить постоянно; это рекомендует используемая библиотека `web-push` : <<https://github.com/web-push-libs/web-push>>.